Remaining Useful Life Prediction for Turbofan Engines Using Machine Learning

Haoshi Wang

Abstract

Predictive maintenance aims to forecast equipment failures before they occur, reducing downtime and maintenance costs. This report presents a machine learning approach to predicting the Remaining Useful Life (RUL) of turbofan engines using the NASA C-MAPSS FD001 dataset. We evaluate three regression models—Linear Regression, Random Forest, and XGBoost—on 100 run-to-failure engine trajectories with 21 sensor measurements. After feature engineering (constant column removal, RUL clipping at 125 cycles, and MinMax normalization), XGBoost achieves the best performance with a test set RMSE of 16.81 cycles and R^2 of 0.824, a 19.3% RMSE reduction over the Linear Regression baseline. Gradient boosting effectively captures sensor degradation patterns for RUL estimation.

1 Introduction

Unplanned equipment failures in industrial systems lead to costly downtime, safety risks, and inefficient maintenance scheduling. Traditional maintenance strategies—reactive (fix after failure) or preventive (fixed-interval servicing)—are either too late or too conservative. *Predictive maintenance* leverages sensor data and machine learning to estimate when a component will fail, enabling just-in-time interventions.

The core task in predictive maintenance is **Remaining Useful Life (RUL) prediction**: given current sensor readings from a machine, estimate how many operational cycles remain before failure. This is a regression problem where the target variable decreases over time as the equipment degrades.

In this work, we use the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) FD001 dataset, a widely used benchmark for RUL prediction. We compare three models of increasing complexity—Linear Regression, Random Forest, and XGBoost—and evaluate their ability to predict engine failure from multivariate sensor time series.

2 Data Description

2.1 Dataset Overview

The C-MAPSS FD001 dataset simulates turbofan engine degradation under a single operating condition and a single fault mode (HPC degradation). Each engine starts in a healthy state and runs until failure. The dataset is split into:

- **Training set**: Complete run-to-failure trajectories for 100 engines.
- **Test set**: Partial trajectories for 100 engines (cut off before failure), with ground-truth RUL values provided separately.

Table 1: FD001 Dataset Statistics

Property	**Value**
Number of engines (train/test)	100 / 100
Total training samples	20,631
Operational settings	3
Sensor measurements	21
Operating conditions	1 (sea level)
Fault modes	1 (HPC degradation)
Min / Max engine lifetime	128 / 362 cycles

2.2 Exploratory Analysis

Figure 1 shows the distribution of engine lifetimes in the training set. Lifetimes range from 128 to 362 cycles, with a roughly uniform distribution, indicating diverse degradation rates across engines.

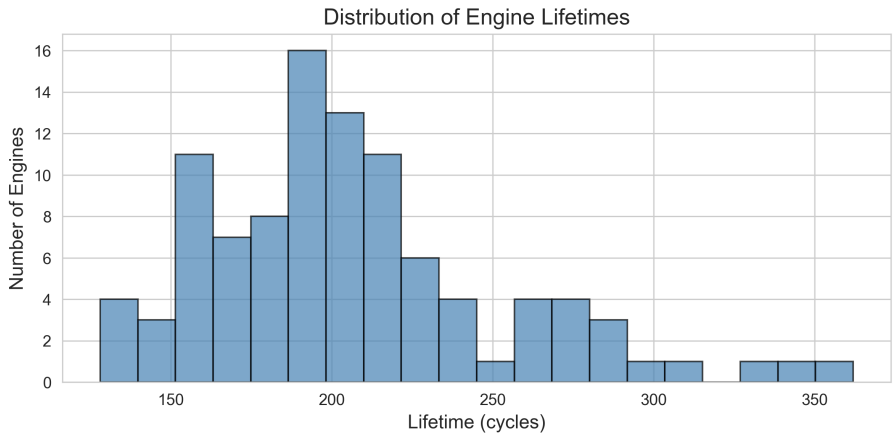


Figure 1: Distribution of engine lifetimes in the FD001 training set.

Not all 21 sensors are equally informative. Figure 2 shows the Pearson correlation of each sensor with RUL. Several sensors (e.g., sensors 2, 3, 4, 11, 15) show strong negative correlations, indicating clear degradation trends. Others (e.g., sensors 1, 5, 10, 16) show near-zero correlation and are effectively constant.

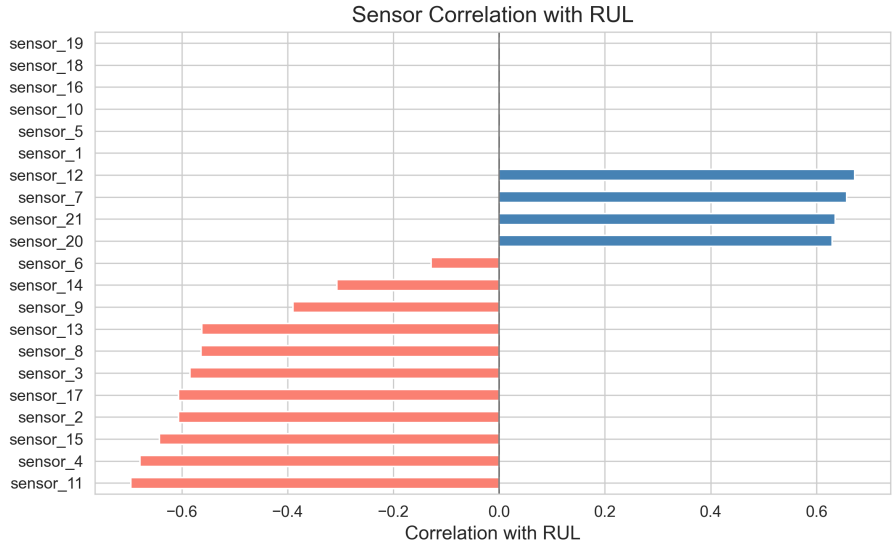


Figure 2: Correlation of each sensor measurement with RUL.

3 Methodology

3.1 Feature Engineering

We apply three preprocessing steps:

1. **Constant column removal**: 8 features with near-zero variance (setting_2, setting_3, sensors 1, 5, 10, 16, 18, 19) are removed, leaving 16 informative features.
2. **RUL clipping at 125 cycles**: Early in an engine's life, degradation is not yet detectable by sensors. Following standard practice, we cap the RUL target at 125, creating a piecewise-linear label (constant at 125 for healthy operation, then linearly decreasing). This focuses model learning on the degradation region. Figure 3 illustrates this transformation.
3. **MinMax normalization**: All features are scaled to $[0, 1]$ to ensure no single sensor dominates due to scale differences.

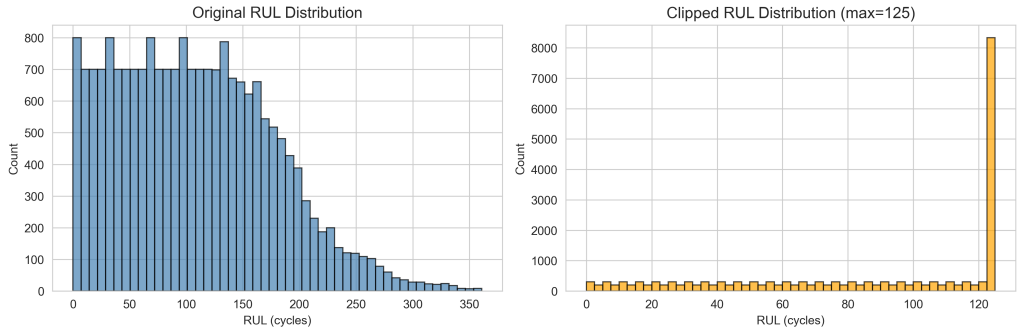


Figure 3: Original vs. clipped RUL distribution. Clipping at 125 cycles concentrates labels in the degradation region.

3.2 Sensor Degradation Patterns

Figure 4 shows the average degradation trends for the six most correlated sensors. Each plot shows the mean sensor value (± 1 standard deviation) across all engines as a function of RUL. Clear monotonic trends near failure (right side) confirm that these sensors carry strong prognostic signals.

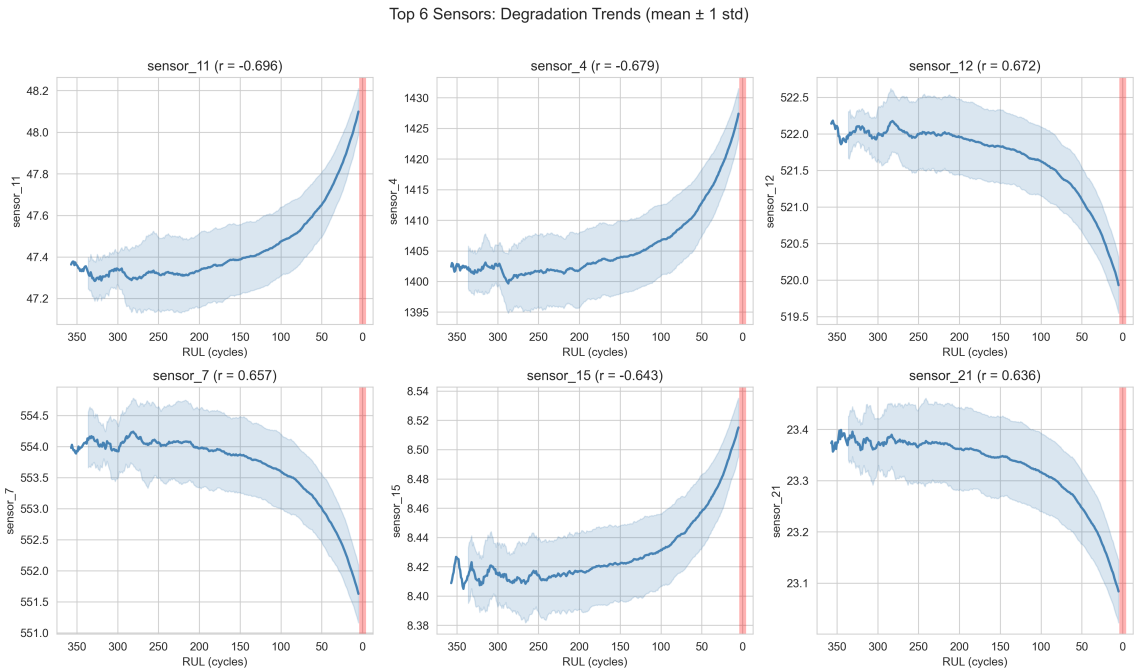


Figure 4: Degradation trends for the top 6 sensors by absolute correlation with RUL. X-axis shows cycles remaining until failure.

3.3 Models

We evaluate three regression models:

- **Linear Regression (LR)**: A baseline that assumes a linear relationship between features and RUL.
- **Random Forest (RF)**: An ensemble of 100 decision trees that captures nonlinear relationships through bagging.
- **XGBoost**: A gradient-boosted tree ensemble (100 trees, learning rate 0.1) that iteratively corrects prediction errors.

For model comparison, training data is split 80/20 **by engine unit** using `GroupShuffleSplit` (80 engines for training, 20 for validation). This prevents data leakage: since consecutive cycles from the same engine are highly autocorrelated, a row-level random split would allow near-identical samples to appear in both train and validation sets, inflating validation metrics. For final test-set evaluation, all models are retrained on the full training data (100 engines, 20,631 samples).

4 Results

4.1 Validation Set Comparison

Table 2 compares the three models on the validation set (20 held-out engines, split by unit via `GroupShuffleSplit`). Both tree-based models substantially outperform Linear Regression, and XGBoost achieves the lowest RMSE and MAE.

Table 2: Validation Set Performance (grouped by engine unit)

Model	RMSE	MAE	R^2
Linear Regression	20.41	16.48	0.761
Random Forest	17.22	12.55	0.830
XGBoost	16.94	12.19	0.835

4.2 Test Set Results

All three models are retrained on the full training data and evaluated on the held-out test set (100 engines, predicting RUL at the last observed cycle, ground truth from `RUL_FD001.txt`). Table 3 shows the results.

Table 3: Test Set Performance (all models, trained on full training data)

Model	RMSE	MAE	R^2
Linear Regression	20.82	16.57	0.730
Random Forest	16.99	11.97	0.820
XGBoost	16.81	11.84	0.824

XGBoost achieves a 19.3% RMSE reduction over Linear Regression on the same test set. The difference between test RMSE (16.81) and validation RMSE (16.94) is small and within normal variation given the sample sizes involved (20 validation engines vs. 100 test engines).

4.3 Feature Importance

Figure 5 shows the XGBoost feature importance. Sensors 11, 4, and 9 are the most important predictors, consistent with their strong correlation with RUL observed in the exploratory analysis.

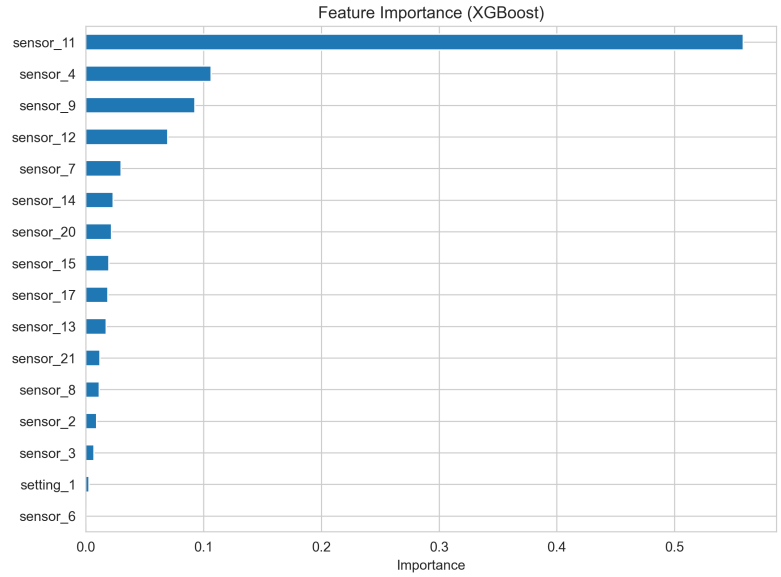


Figure 5: XGBoost feature importance ranking.

4.4 Predicted vs. Actual RUL

Figure 6 shows the predicted vs. actual RUL for the 100 test engines. Points cluster near the diagonal, indicating good prediction accuracy. The model performs well across the full RUL range, with slightly more scatter at high RUL values where degradation signals are weaker.

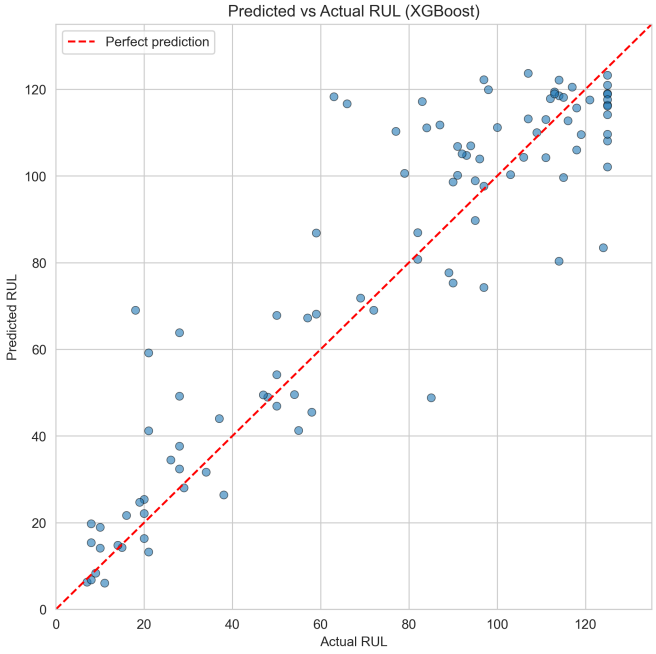


Figure 6: Predicted vs. actual RUL on the test set. The dashed red line indicates perfect prediction.

5 Conclusion

This study demonstrates that machine learning models can effectively predict turbofan engine RUL from multivariate sensor data. Key findings:

- XGBoost achieves the best test-set performance (RMSE = 16.81, $R^2 = 0.824$), a 19.3% RMSE reduction over the Linear Regression baseline (RMSE = 20.82), with all models evaluated on the same test set.
- Simple preprocessing—removing constant sensors, clipping RUL at 125, and MinMax normalization—is sufficient for strong baseline performance.
- Splitting the validation set by engine unit (rather than by row) prevents data leakage and gives more realistic validation estimates.
- Sensors 11, 4, and 9 are the most important features, showing clear degradation trends as engines approach failure.

Limitations. The FD001 dataset represents a single operating condition and fault mode. Real-world engines operate under varying conditions (addressed by FD002–FD004 datasets). The current approach treats each time step independently, ignoring temporal dependencies within engine trajectories.

Future work. Recurrent neural networks (e.g., LSTM) or temporal convolutional networks could capture sequential degradation patterns. Extending the analysis to multi-condition datasets (FD002–FD004) would test model robustness. Additionally, incorporating a scoring function that penalizes late predictions more heavily than early ones would better reflect real-world maintenance priorities.